

Домашнее задание № 14

Набор запросов для стандартного дашборда:

```
{
  "params": {
    "rounding": "60",
    "seconds": "86400",
    "from": "",
    "to": ""
  },
  "queries": [
    {
      "title": "Failed Queries/sec",

      "query": "WITH toDateTimeOrDefault({from:String}, '', now()) -
{seconds:UInt32}) AS from,\n  toDateTimeOrDefault({to:String}, '', now()) AS
to\nSELECT toStartOfInterval(event_time, INTERVAL {rounding:UInt32}
SECOND)::INT AS t, avg(ProfileEvent_FailedQuery)\nFROM merge('system',
'^metric_log')\nWHERE event_date BETWEEN toDate(from) AND toDate(to) AND
event_time BETWEEN from AND to\nGROUP BY t\nORDER BY t WITH FILL STEP
{rounding:UInt32}"

    },
    {
      "title": "Query Duration p95 (ms)",

      "query": "WITH toDateTimeOrDefault({from:String}, '', now()) -
{seconds:UInt32}) AS from,\n  toDateTimeOrDefault({to:String}, '', now()) AS
to\nSELECT toStartOfInterval(event_time, INTERVAL {rounding:UInt32}
SECOND)::INT AS t, quantile(0.95)(query_duration_ms)\nFROM
system.query_log\nWHERE event_date BETWEEN toDate(from) AND toDate(to)
AND event_time BETWEEN from AND to AND type = 'QueryFinish'\nGROUP BY
t\nORDER BY t WITH FILL STEP {rounding:UInt32}"

    },
    {
      "title": "Query Duration p99 (ms)",

      "query": "WITH toDateTimeOrDefault({from:String}, '', now()) -
{seconds:UInt32}) AS from,\n  toDateTimeOrDefault({to:String}, '', now()) AS
to\nSELECT toStartOfInterval(event_time, INTERVAL {rounding:UInt32}
SECOND)::INT AS t, quantile(0.99)(query_duration_ms)\nFROM
system.query_log\nWHERE event_date BETWEEN toDate(from) AND toDate(to)
AND event_time BETWEEN from AND to AND type = 'QueryFinish'\nGROUP BY
t\nORDER BY t WITH FILL STEP {rounding:UInt32}"

    }
  ]
}
```

},

{

"title": "Memory Resident (bytes)",

"query": "WITH toDateTimeOrDefault({from:String}, '', now() -
{seconds:UInt32}) AS from,\n toDateTimeOrDefault({to:String}, '', now()) AS
to\nSELECT toStartOfInterval(event_time, INTERVAL {rounding:UInt32}
SECOND)::INT AS t, avg(value)\nFROM merge('system',
'^asynchronous_metric_log')\nWHERE event_date BETWEEN toDate(from) AND
toDate(to) AND event_time BETWEEN from AND to AND metric =
'MemoryResident'\nGROUP BY t\nORDER BY t WITH FILL STEP
{rounding:UInt32}"

},

{

"title": "Network Traffic (bytes/sec)",

"query": "WITH toDateTimeOrDefault({from:String}, '', now() -
{seconds:UInt32}) AS from,\n toDateTimeOrDefault({to:String}, '', now()) AS
to\nSELECT toStartOfInterval(event_time, INTERVAL {rounding:UInt32}
SECOND)::INT AS t,\n avg(ProfileEvent_NetworkReceiveBytes) AS received,\n avg(ProfileEvent_NetworkSendBytes) AS sent\nFROM merge('system',
'^metric_log')\nWHERE event_date BETWEEN toDate(from) AND toDate(to) AND
event_time BETWEEN from AND to\nGROUP BY t\nORDER BY t WITH FILL STEP
{rounding:UInt32}"

},

{

"title": "Background Merge Tasks",

"query": "WITH toDateTimeOrDefault({from:String}, '', now() -
{seconds:UInt32}) AS from,\n toDateTimeOrDefault({to:String}, '', now()) AS
to\nSELECT toStartOfInterval(event_time, INTERVAL {rounding:UInt32}
SECOND)::INT AS t,
max(CurrentMetric_BackgroundMergesAndMutationsPoolTask)\nFROM merge('system', '^metric_log')\nWHERE event_date BETWEEN toDate(from) AND
toDate(to) AND event_time BETWEEN from AND to\nGROUP BY t\nORDER BY t
WITH FILL STEP {rounding:UInt32}"

},

{

"title": "Load Average (1/5/15 min)",

"query": "WITH toDateTimeOrDefault({from:String}, '', now() -
{seconds:UInt32}) AS from,\n toDateTimeOrDefault({to:String}, '', now()) AS
to\nSELECT toStartOfInterval(event_time, INTERVAL {rounding:UInt32}
SECOND)::INT AS t,\n maxIf(value, metric = 'LoadAverage1') AS la1,\n maxIf(value, metric = 'LoadAverage5') AS la5,\n maxIf(value, metric =
'LoadAverage15') AS la15\nFROM merge('system', '^asynchronous_metric_log')
\nWHERE event_date BETWEEN toDate(from) AND toDate(to) AND event_time

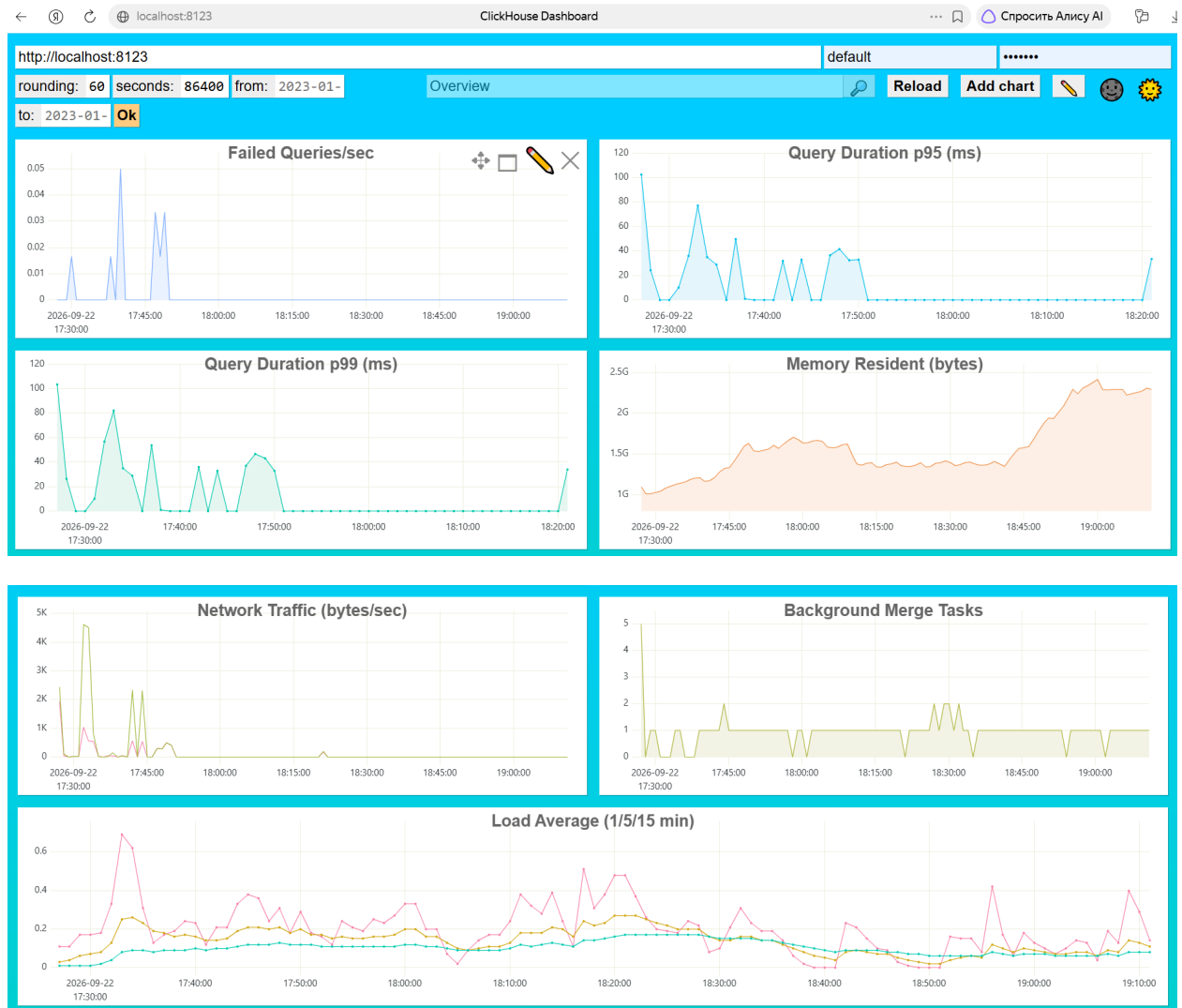
```
BETWEEN from AND to\n  AND metric IN ('LoadAverage1', 'LoadAverage5',  
'LoadAverage15')\nGROUP BY t\nORDER BY t WITH FILL STEP  
{rounding:UInt32}"
```

```
}
```

```
]
```

```
}
```

Скриншот дашборда:



Задание со * (необязательное)

1. Настройте таблицу с логами с движком Engine=NULL

```
--Таблица с логами с движком Engine=NULL
CREATE TABLE IF NOT EXISTS logs_null
(
  event_time DateTime,
  level LowCardinality(String),
  message String,
  service LowCardinality(String),
  trace_id String
)
ENGINE = Null;
```

2. Создайте дополнительную реплицируемую таблицу такой же структуры с материализованным полем для идентификации реплики.

```
--Реплицируемая таблица с материализованным полем для идентификации реплики
CREATE TABLE IF NOT EXISTS logs_replicated
(
    event_time DateTime,
    level LowCardinality(String),
    message String,
    service LowCardinality(String),
    trace_id String,
    replica_name String MATERIALIZED hostName()
)
ENGINE = ReplicatedMergeTree('/clickhouse/tables/{shard}/logs_replicated', '{replica}')
PARTITION BY toYYYYMM(event_time)
ORDER BY (service, event_time);
```

3. Настройте материализованное представление (MV) для переноса логов из таблицы с Engine=NULL в реплицируемую таблицу.

```
--Материализованное представление для переноса логов
CREATE MATERIALIZED VIEW IF NOT EXISTS mv_logs_to_replicated
TO logs_replicated
AS SELECT
    event_time,
    level,
    message,
    service,
    trace_id
FROM logs_null;
```

4. Вставка тестовых данных

```
-- вставка данных на первой реплике
INSERT INTO logs_null VALUES
(now(), 'INFO', 'Test message', 'auth-service', 'trace-001'),
(now(), 'ERROR', 'Another error', 'payment-service', 'trace-002');
```

5. Проверка репликации

```
-- проверка на второй реплике
SELECT *, replica_name FROM logs_replicated ORDER BY event_time;
```

logs_replicated 1 X

SELECT *, replica_name FROM logs_replicated ORDER BY event_time; Введите SQL выражение чтобы отфильтровать результаты

	event_time	level	message	service	trace_id	replica_name
1	2026-09-22 22:37:19	INFO	Test message	auth-service	trace-001	clickhouse-replica-1
2	2026-09-22 22:37:19	ERROR	Another error	payment-service	trace-002	clickhouse-replica-1

6. Секции конфигурации

 keeper.xml – Блокнот

Файл Правка Формат Вид Справка

```
<clickhouse>
  <zookeeper>
    <node>
      <host>zookeeper</host>
      <port>2181</port>
    </node>
  </zookeeper>
</clickhouse>
```

 replica-1-macros.xml – Блокнот

Файл Правка Формат Вид Справка

```
<clickhouse>
  <macros>
    <shard>1</shard>
    <replica>replica-1</replica>
  </macros>
</clickhouse>
```

 replica-2-macros.xml – Блокнот

Файл Правка Формат Вид Справка

```
<clickhouse>
  <macros>
    <shard>1</shard>
    <replica>replica-2</replica>
  </macros>
</clickhouse>
```

```
# Первая реплика ClickHouse
clickhouse-replica-1:
  image: clickhouse/clickhouse-server:latest
  container_name: clickhouse-replica-1
  hostname: clickhouse-replica-1
  restart: unless-stopped

  ports:
    - "8123:8123"      # HTTP
    - "9000:9000"      # TCP
    - "9009:9009"      # interserver

  environment:
    - CLICKHOUSE_DB=${CLICKHOUSE_DB:-default}
    - CLICKHOUSE_USER=${CLICKHOUSE_USER:-default}
    - CLICKHOUSE_PASSWORD=${CLICKHOUSE_PASSWORD}
    - CLICKHOUSE_LOG_LEVEL=information

  volumes:
    - ./config/listen.xml:/etc/clickhouse-server/config.d/listen.xml
    - ./config/optimized_config.xml:/etc/clickhouse-server/config.d/optimized_config.xml
    - ./config/replica-1-macros.xml:/etc/clickhouse-server/config.d/macros.xml # HOBOE
    - ./config/keeper.xml:/etc/clickhouse-server/config.d/keeper.xml          # HOBOE
    - ./user_files:/var/lib/clickhouse/user_files/
    - clickhouse-data-1:/var/lib/clickhouse
    - clickhouse-logs-1:/var/log/clickhouse-server

  ulimits:
    nofile:
      soft: 262144
      hard: 262144

  depends_on:
    - zookeeper

  networks:
    - clickhouse-network

# Вторая реплика ClickHouse
clickhouse-replica-2:
  image: clickhouse/clickhouse-server:latest
  container_name: clickhouse-replica-2
  hostname: clickhouse-replica-2
  restart: unless-stopped

  ports:
    - "8124:8123"      # HTTP (другой порт)
    - "9001:9000"      # TCP (другой порт)
    - "9010:9009"      # interserver (другой порт)

  environment:
    - CLICKHOUSE_DB=${CLICKHOUSE_DB:-default}
    - CLICKHOUSE_USER=${CLICKHOUSE_USER:-default}
    - CLICKHOUSE_PASSWORD=${CLICKHOUSE_PASSWORD}
    - CLICKHOUSE_LOG_LEVEL=information

  volumes:
    - ./config/listen.xml:/etc/clickhouse-server/config.d/listen.xml
    - ./config/optimized_config.xml:/etc/clickhouse-server/config.d/optimized_config.xml
    - ./config/replica-2-macros.xml:/etc/clickhouse-server/config.d/macros.xml # HOBOE
    - ./config/keeper.xml:/etc/clickhouse-server/config.d/keeper.xml          # HOBOE
    - ./user_files:/var/lib/clickhouse/user_files/
    - clickhouse-data-2:/var/lib/clickhouse
    - clickhouse-logs-2:/var/log/clickhouse-server

  ulimits:
    nofile:
      soft: 262144
      hard: 262144

  depends_on:
    - zookeeper

  networks:
    - clickhouse-network
```